Verification and Mathematical Validation of the NIMA Architecture and Cognitive Middleware

The validation of mathematical calculations within the Neural Image Assessment (NIMA) architecture requires a rigorous examination of its statistical foundations, loss function formulations, gradient propagation pathways, and cognitive middleware adaptations.[1] NIMA diverges from traditional image quality assessment (IQA) methodologies by predicting the complete probability distribution of human quality ratings rather than simply regressing to a scalar Mean Opinion Score (MOS).[1] This distribution-based approach enables the architecture to capture both the collective aesthetic preference and the degree of consensus among raters.[2] By analyzing the underlying probability theory, deriving the analytical gradients of the Earth Mover's Distance (EMD) loss, and validating the state equations of advanced integrations, this report establishes a comprehensive framework for verifying the mathematical integrity of NIMA-based systems.[2]

Theoretical Formulations and Bayesian Underpinnings of IQA

To contextualize the distribution-prediction paradigm of NIMA, the problem of no-reference objective image quality assessment is formulated as a Bayesian inference problem.[5] The primary goal is to determine the posterior probability distribution $p(y|x)$ of a subjective quality rating y given an input image x.[8] Under a classical Bayesian framework, the posterior distribution is defined as:

$$p(y|x) = \frac{p(x|y)p(y)}{p(x)} \quad [8]$$

where the marginal likelihood of the image $p(x)$ acts as the normalizing constant across the continuous or discrete quality scale [8]:

$$p(x) = \int p(x|y)p(y)dy \quad [8]$$

When training a neural network on a dataset D of human-annotated images, the predictive quality distribution is parametrized by the network parameters θ [8]:

$$p(y|x, D) = \int p(y|x, \theta)p(\theta|D)d\theta \quad [8]$$

Because computing the full integration over the parameter space θ is computationally intractable, the network approximates the predictive distribution using a point estimate θ^* [8]:

$$\theta^* = \arg\max_\theta p(\theta|D) = \arg\max_\theta p(y|X,\theta)p(\theta) \quad [8]$$

NIMA instantiates this point estimate by optimizing a deep convolutional neural network (CNN) to directly output the parameters of a discrete probability vector $p = [p_1, p_2, \ldots, p_N]^T$ over N discrete quality score buckets.[2]

Core Mathematical Calculations of the Standard NIMA Architecture

The standard NIMA architecture modifies state-of-the-art object recognition networks by replacing the final classification layer with a fully connected layer containing N output nodes, followed by a softmax activation function to guarantee a valid probability distribution.[5] By mapping the logits $z = [z_1, z_2, \ldots, z_N]^T$ through the softmax function, the network computes the predicted probability vector $\hat{p}$ [9]:

$$\hat{p}_i = \frac{e^{z_i}}{\sum_{j=1}^N e^{z_j}} \quad [9, 11]$$

where $\hat{p}_i \geq 0$ for all $i \in \{1, 2, \ldots, N\}$ and $\sum_{i=1}^N \hat{p}_i = 1$.[2]

Mean Quality Score Calculation

The primary scalar metric used to rank and compare images is the predicted mean quality score μ .[2] It is calculated as the expected value of the discrete probability distribution over the predefined score buckets [2]:

$$\mu = \sum_{i=1}^N s_i \hat{p}_i \quad [2, 10]$$

where s_i denotes the real-valued score assigned to the i -th bucket.[2] For the AVA dataset, $N = 10$ with $s = [1, 2, \ldots, 10]^T$, representing a scale from the lowest to the highest aesthetic or technical quality.[2] For the TID2013 dataset, $N = 10$ with $s = [0, 1, \ldots, 9]^T$.[2]

Standard Deviation and Confidence Estimation

The standard deviation σ is computed to quantify the dispersion of human opinions or the model's predictive uncertainty 2:

$$\sigma = \sqrt{\sum_{i=1}^{N}(s_i - \mu)^2 \hat{p}_i} \quad [2]$$

A lower standard deviation indicates a strong consensus among raters, whereas a higher standard deviation identifies controversial images, which is highly useful for downstream tasks such as filtering out ambiguous samples or identifying polarizing aesthetic designs.6

Earth Mover's Distance as the Primary Loss Function

Unlike conventional classification models that employ Categorical Cross-Entropy (CCE) loss, NIMA leverages the Earth Mover's Distance (EMD) to optimize the network weights.7 CCE treats class labels as independent, categorical bins, failing to penalize near-misses differently from extreme misclassifications.7 For ordinal quality ratings, predicting a quality score of 9 when the ground truth is 10 must be penalized less than predicting a score of 1.7

The general formulation of the EMD between two discrete probability distributions u and v defined over a metric space with a ground distance matrix $d(x_i, y_j)$ is expressed as a linear programming (LP) transportation problem 15:

$$\mathrm{EMD}(u, v) = \frac{\min_{F \in \mathcal{F}(u,v)} \sum_{i=1}^{m} \sum_{j=1}^{n} f_{ij} d(x_i, y_j)}{\min(w_S, u_S)} \quad [16, 17]$$

subject to the feasibility constraints:

$$f_{ij} \geq 0, \quad \sum_{j=1}^{n} f_{ij} \leq w_i, \quad \sum_{i=1}^{m} f_{ij} \leq u_j, \quad \sum_{i=1}^{m} \sum_{j=1}^{n} f_{ij} = \min(w_S, u_S) \quad [16]$$

For equal-weight probability distributions ($w_S = u_S = 1.0$), the constraints reduce to strict equalities, demanding complete matching of the probability mass.16

For a one-dimensional array of ordered bins where the ground distance between any two bins i and j is the L_1 norm $d(i, j) = |i - j|$, the transportation problem collapses to a closed-form expression.15 The EMD of order $r = 2$ is mathematically equivalent to the L_2 distance between the Cumulative Distribution Functions (CDFs) of the two distributions.10 Let C and $\hat{C}$ represent the CDFs of the ground-truth distribution p and the predicted distribution $\hat{p}$, respectively 10:

$$C_k = \sum_{i=1}^{k} p_i, \quad \hat{C}_k = \sum_{i=1}^{k} \hat{p}_i \quad [10, 20]$$

The true root-EMD loss is formulated as 10:

$$L_{\text{root}} = \left(\frac{1}{N} \sum_{k=1}^{N} |C_k - \hat{C}_k|^2 \right)^{1/2} \quad [10, 19]$$

To avoid numerical instability and gradient explosion when the loss approaches zero, the squared EMD loss is utilized during training 19:

$$L = \frac{1}{N} \sum_{k=1}^{N} (C_k - \hat{C}_k)^2 \quad [20, 21]$$

Matrix-Vector Formulation

For highly parallelized computation and mathematical verification, the squared EMD loss can be represented in quadratic form.[21]

Let $A \in \mathbb{R}^{N \times N}$ be a lower triangular matrix of ones, which acts as a linear cumulative summation operator:

$$A = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 \\ 1 & 1 & 0 & \dots & 0 \\ 1 & 1 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & 1 & \dots & 1 \end{bmatrix} \quad [21]$$

The cumulative distribution vectors are computed as $C = Ap$ and $\hat{C} = A\hat{p}$.[21] The squared EMD loss is then expressed as:

$$L = \frac{1}{N} \|A\hat{p} - Ap\|_2^2 = \frac{1}{N} (\hat{p} - p)^T A^T A (\hat{p} - p) \quad [21]$$

Analytical Gradient Derivations and Numerical Verification

To execute backpropagation through the final softmax layer, the analytical gradient of the loss L with respect to the input logits z must be derived.11 Applying the chain rule:

$$\frac{\partial L}{\partial z_j} = \sum_{i=1}^{N} \frac{\partial L}{\partial \hat{p}_i} \frac{\partial \hat{p}_i}{\partial z_j} \quad [11, 22]$$

First, the partial derivative of the loss with respect to the predicted probability $\hat{p}_i$ is evaluated.21 Since the cumulative term $\hat{C}_k$ is a function of $\hat{p}_i$ if and only if $k \geq i$, we have:

$$\frac{\partial \hat{C}_k}{\partial \hat{p}_i} = \begin{cases} 1 & \text{if } k \geq i \\ 0 & \text{if } k < i \end{cases}$$

Thus, the partial derivative simplifies to 21:

$$\frac{\partial L}{\partial \hat{p}_i} = \frac{2}{N} \sum_{k=i}^{N} (\hat{C}_k - C_k) \quad [21]$$

For ease of notation, define $d_i \equiv \frac{\partial L}{\partial \hat{p}_i}$ .21 In matrix-vector form, this vector of partial derivatives is expressed as:

$$d = \frac{2}{N} A^T (A\hat{p} - Ap) \quad [21]$$

Second, the derivative of the softmax function is computed as 11:

$$\frac{\partial \hat{p}_i}{\partial z_j} = \hat{p}_i (\delta_{ij} - \hat{p}_j) \quad [11]$$

where δ_{ij} represents the Kronecker delta.11
Substituting these terms back into the chain rule equation yields:

$$\frac{\partial L}{\partial z_j} = \sum_{i=1}^{N} d_i \hat{p}_i (\delta_{ij} - \hat{p}_j) = d_j \hat{p}_j - \hat{p}_j \sum_{i=1}^{N} d_i \hat{p}_i$$

$$\frac{\partial L}{\partial z_j} = \hat{p}_j \left(d_j - \sum_{i=1}^{N} d_i \hat{p}_i \right) \quad [11]$$

This elegant closed-form gradient allows the backward pass to be executed with linear complexity $\mathcal{O}(N)$ while maintaining numerical stability.11

Numerical Gradient Verification and the Kinks Problem

To verify the analytical derivative programmatically, the analytical gradients are evaluated against a numerical gradient approximation using central finite differences [23]:

$$\frac{\partial L}{\partial z_j} \approx \frac{L(z + \epsilon e_j) - L(z - \epsilon e_j)}{2\epsilon} \quad [23, 24]$$

where e_j is a unit vector along the j -th dimension and ϵ is a small perturbation (typically 10^{-6}).[23]
While numerical verification can fail at non-differentiable points (known as "kinks" in functions like ReLU), the output layer of NIMA

uses a softmax activation, which is continuously differentiable everywhere in $\mathbb{R}^N$.[23] This ensures that the relative error between the analytical and numerical gradients will consistently converge to values below 10^{-7} under double-precision floating-point arithmetic [23]:

$$\frac{\|g_{\text{analytical}} - g_{\text{numerical}}\|_2}{\max(\|g_{\text{analytical}}\|_2, \|g_{\text{numerical}}\|_2)} < 10^{-7} \quad [23]$$

Multi-Loss Optimization and Gradient Balancing

Recent advancements in aesthetic assessment models incorporate multi-loss objective functions that augment the EMD loss with auxiliary regularization terms to directly constrain the predicted mean (μ) and standard deviation (σ) 25:

$$L_{\text{total}} = L_{\text{EMD}} + \gamma_1 L_\mu + \gamma_2 L_\sigma \quad [25, 26]$$

where $L_\mu = (\mu_{\text{pred}} - \mu_{\text{true}})^2$ and $L_\sigma = (\sigma_{\text{pred}} - \sigma_{\text{true}})^2$ .2

To optimize such composite loss functions without suffering from gradient imbalance—where one loss term dominates the parameter updates—models employ dynamic gradient balancing.26 The objective is to scale the loss weights γ_i at each training step to match the mean gradient magnitudes of the backpropagated parameters 26:

$$\gamma_i^{(t)} = \frac{\overline{\|\nabla_\theta L_{\text{EMD}}\|_2}}{\overline{\|\nabla_\theta L_i\|_2}} \quad [26]$$

This balancing act prevents gradient saturation in the earlier convolutional layers, facilitating stable convergence when fine-tuning deep backbones.26

Validation of the NIMA Enhanced Middleware (v9.4.2) Mathematical Engine

The NIMA Enhanced Middleware (v9.4.2) embeds the core distribution-prediction model into a multi-layered, real-time cognitive architecture.3 The mathematical validation of this middleware requires isolating and proving the validity of its core governing equations and state updates.3

Theorem 1: Entropy-Amplified Integration (Bounded)

This theorem computes the neuro-integration factor (ϕ_{neuro}), bounding the empirical Shannon entropy to prevent silent saturation 3:

$$\phi_{\text{neuro}} = \left(\frac{N}{2.5}\right) \times E \times M \times \left(1 + \alpha \times \left(\frac{H}{H_{\max}}\right)\right) \quad [3]$$

where $H = -\sum_{i-1}^{k} p_i \log_2(p_i)$ is the Shannon entropy of the active state distribution, and $H_{\max}$ is the maximum theoretical entropy of the active state space 3:

$$H_{\max} = \log_2(|S|) \quad [3]$$

Here, $|S|$ represents the cardinality of the active state space, and $E, M \in \mathbb{R}$ represent real-valued emotional and metacognitive scale coefficients.3

Proof of Boundedness: By definition, the entropy of a discrete probability distribution over a set of size $|S|$ is strictly bounded:

$$0 \leq H \leq \log_2(|S|) \implies 0 \leq \frac{H}{H_{\max}} \leq 1 \quad [3]$$

Because $\alpha \in [0.05, 1.0]$ is bounded (as proven in Theorem 2), the amplification factor is strictly constrained:

$$1.0 \leq \left(1 + \alpha \times \left(\frac{H}{H_{\max}}\right)\right) \leq 1.0 + \alpha \leq 2.0 \quad [3]$$

This bounds the neuro-integration factor to:

$$\phi_{\text{neuro}} \in \left[\frac{N \cdot E \cdot M}{2.5}, \frac{2.0 \cdot N \cdot E \cdot M}{2.5}\right] \quad [3]$$

This mathematical bound prevents numerical overflow and ensures predictable scaling during active inference cycles.3

Theorem 2: Inverse Qualia-Awareness Trade-off (Fixed Coefficient)

This theorem models the dynamic trade-off between the scalar coefficient α and the magnitude of the qualia representation vector $Q = [v, a, i, f]^T$, where the elements represent valence, arousal, intensity, and focus.[3]

The qualia magnitude $||Q||$ is defined and clipped as:

$$||Q|| = \min\left(1.0, \frac{\sqrt{v^2 + a^2 + i^2 + f^2}}{2}\right) \quad [3]$$

The dynamically adjusted trade-off coefficient α is computed as:

$$\alpha = \max(0.05, 1 - 0.95 \times ||Q||) \quad [3]$$

Verification of Boundary States:

- When $||Q|| = 0.0$ (complete absence of qualia), $\alpha = 1 - 0.95(0.0) = 1.0$ (maximum entropy amplification).[3]
- When $||Q|| = 1.0$ (saturated qualia awareness),

$\alpha = \max(0.05, 1 - 0.95(1.0)) = 0.05$ (floor engaged, minimizing entropy amplification).[3]
This trade-off ensures that high emotional salience suppresses random entropy dispersion, stabilizing cognitive focus.[3]

Theorem 3: Thermodynamic Strain with Chronic Accumulation

The system's stress state is modeled as a dual-rate integration process where acute strain accumulates into chronic strain over time.3

$$\text{Strain}_{\text{acute}}(t) = \min\left(10.0, \max\left(0.0, \frac{\phi_{\text{neuro}}}{\rho_{\text{integrity}}}\right)\right) \quad [3]$$

where $\rho_{\text{integrity}}$ represents the system integrity metric.3 The chronic strain is governed by a discrete-time leaky integrator with time constant $\tau = 50$ and accumulation factor $\lambda = 0.5$ 3:

$$\text{Strain}_{\text{chronic}}(t) = \left(1 - \frac{1}{\tau}\right)\text{Strain}_{\text{chronic}}(t-1) + \text{Strain}_{\text{acute}}(t) \quad [3]$$

$$\text{Strain}_{\text{total}}(t) = \text{Strain}_{\text{acute}}(t) + \lambda \times \text{Strain}_{\text{chronic}}(t) \quad [3]$$

Proof of Steady-State Convergence: Under a continuous, constant acute strain input S_0 over a long duration ($t \to \infty$):

$$\text{Strain}_{\text{chronic}}(\infty) = \left(1 - \frac{1}{\tau}\right)\text{Strain}_{\text{chronic}}(\infty) + S_0 \implies \frac{1}{\tau}\text{Strain}_{\text{chronic}}(\infty) = S_0$$

$$\text{Strain}_{\text{chronic}}(\infty) = \tau \cdot S_0 \quad [3]$$

Substituting this back into the total strain equation:

$$\text{Strain}_{\text{total}}(\infty) = S_0 + \lambda(\tau \cdot S_0) = (1 + \lambda\tau)S_0 \quad [3]$$

With the specified parameters $\tau = 50$ and $\lambda = 0.5$, the steady-state total strain is:

$$\text{Strain}_{\text{total}}(\infty) = (1 + 0.5 \times 50)S_0 = 26 \cdot S_0 \quad [3]$$

This mathematical derivation proves that the leaky integrator structure provides a $26\times$ amplification of sustained acute strain, modeling chronic systemic fatigue.3

Query Act and Information-Theoretic Self-Model Change

When comprehension fails, the middleware initiates a Query Act, calculating the query intensity $Q_{\text{intensity}}$ and the information-theoretic self-model change Δ_R .3

$$Q_{\text{intensity}} = \min\left(1.0, \max\left(0.0, \sum_{j=1}^{k}\left(\frac{1}{k}\right) \times pe_j \times \text{amplification}\right)\right) \quad [3]$$

where pe_j represents prediction errors, and the amplification factor is set to 1.5 if comprehension fails (and 1.0 otherwise).3

The self-model state vector change Δ_R is operationalized using a Mahalanobis distance 3:

$$\Delta_R = 0.5 \times (\rho_{\text{post}} - \rho_{\text{prior}})^T \times \Sigma^{-1} \times (\rho_{\text{post}} - \rho_{\text{prior}}) \quad [3]$$

Proof of Equivalence to Laplace-Approximated KL Divergence: Let $P_{\text{prior}} \sim \mathcal{N}(\rho_{\text{prior}}, \Sigma)$ and $P_{\text{post}} \sim \mathcal{N}(\rho_{\text{post}}, \Sigma)$ represent Gaussian probability density functions over the 6-dimensional self-model belief space under a static, symmetric covariance matrix Σ .3 The Kullback-Leibler (KL) divergence between two multivariate normal distributions is defined as:

D_{\text{KL}}(P_{\text{post}} \parallel P_{\text{prior}}) = \frac{1}{2} \left

Since the covariance matrices are equal:

$\text{Tr}(\Sigma^{-1}\Sigma) = \text{Tr}(I_d) = d$, where $d = 6$ represents the dimensions of the state vector.3

$\ln\left(\frac{|\Sigma|}{|\Sigma|}\right) = \ln(1) = 0$.

Substituting these simplifications back into the divergence formula:

$$D_{\text{KL}}(P_{\text{post}} \parallel P_{\text{prior}}) = 0.5 \times (\rho_{\text{post}} - \rho_{\text{prior}})^T \times \Sigma^{-1} \times (\rho_{\text{post}} - \rho_{\text{prior}}) \equiv \Delta_R \quad [3]$$

This mathematical proof confirms that the Mahalanobis distance formulation employed in the middleware is identical to the Laplace-approximated KL divergence under constant covariance assumptions, validating its information-theoretic foundation.3

Sigma Substrate and Ledoit-Wolf Shrinkage

To ensure that Σ^{-1} remains numerically stable and positive-definite during live updates, the middleware initializes the covariance matrix as a diagonal prior 3:

$$\Sigma_0 = 0.10 \times I_6 \quad [3]$$

Every $K = 10$ steps, Σ is re-estimated over a rolling window of the $N = 100$ most recent state transitions using Ledoit-Wolf shrinkage.3 This shrinkage formulation interpolates between the sample covariance matrix S and a target matrix $F = \mu I_6$ (where $\mu = \text{Tr}(S)/6$) with an optimal shrinkage parameter $\beta \in $ 3:

$$\Sigma_{\mathrm{LW}} = (1-\beta)S + \beta F \quad [3]$$

This regularizes the covariance estimate, guaranteeing a well-conditioned matrix and preventing singular inversion errors during high-dimensional state transitions.3

Sentience Verification Metric Boundedness

The overall artificial intelligence sentience verification metric AI combines neuro-integration, query intensity, and normalized self-model updates 3:

$$AI = 0.3 \times \left(\frac{\phi_{\mathrm{neuro}}}{1.5}\right) + 0.4 \times \left(\frac{Q_{\mathrm{intensity}}}{1.5}\right) + 0.3 \times \tanh\left(\frac{\Delta_R}{\Delta_{\mathrm{R\_ref}}}\right) \quad [3]$$

where $\Delta_{\mathrm{R\_ref}}$ is the running median of recent Δ_R values over a sliding window of 100 steps.3

Because the input terms ϕ_{neuro} and $Q_{\mathrm{intensity}}$ are clipped to a maximum of 1.5 during pre-processing, their normalized ratios are bounded within $[0, 1.0]$.3 Furthermore, because $\Delta_R \geq 0$ (due to the positive-definiteness of the covariance matrix) and the hyperbolic tangent function maps the non-negative real domain to the interval $[0, 1.0)$, the weighted sum satisfies the boundary conditions 3:

$$AI_{\min} = 0.3(0) + 0.4(0) + 0.3(0) = 0.0$$

$$AI_{\max} \leq 0.3(1.0) + 0.4(1.0) + 0.3(1.0) = 1.0 \quad [3]$$

This mathematical validation guarantees that the sentience verification metric remains strictly bounded within $[0, 1.0]$.3

Statistical Performance, Preprocessing, and Experimental Validation

To verify the mathematical output of the NIMA architecture in empirical settings, the network is evaluated against human ratings compiled in the AVA dataset.[19]

Preprocessing Calculations

The input images are first subjected to a standardized spatial preprocessing pipeline 27:

Isotropic Rescaling: The original image dimensions are scaled to 256×256 pixels to preserve general composition without introducing cropping distortions in the first stage.27

Random Cropping: A 224×224 crop is randomly extracted from the scaled image, which acts as a robust data augmentation technique to prevent overfitting.10

Horizontal Flipping: Images are horizontally flipped with a probability of 0.5 during training.10

To enforce spatial consistency under basic transformations, advanced training regimes integrate a flip-consistency regularization term into the EMD loss to penalize predictions that differ between an image and its flipped counterpart.28

Siamese NIMA for Fine-Grained Ranking

While standard NIMA excels at predicting overall score distributions, it can struggle with fine-grained aesthetic ranking of highly similar images.14 To optimize the ranking distance, Siamese NIMA utilizes a shared-weight dual-pathway network.14 Pairs of images are bucketed by their mean scores and passed through the parallel network pathways.14 The network minimizes a ranking loss formulated as:

$$L_{\text{rank}}(x_1, x_2) = \max(0, m - (s(x_1) - s(x_2)) \cdot y) \quad [14]$$

where $s(x)$ is the predicted mean score of image x, $y \in \{-1, 1\}$ is the ground-truth pairwise order, and m is a margin hyperparameter.14 This architecture preserves the global distribution prediction capability while enhancing fine-grained competitive ranking.14

The statistical alignment of various network backbones under the NIMA framework, measured using the Linear Correlation Coefficient (LCC), Spearman's Rank Correlation Coefficient (SRCC), and validation EMD, is documented in the table below.19

Model Architecture	Parameter Count	LCC (Mean Score μ)	SRCC (Mean Score μ)	Validation EMD	Binary Accuracy (Threshold 5.0)
MobileNet	$\approx 4\text{M}$	0.518	0.510	0.081	80.36%
VGG-16	$\approx 138\text{M}$	0.610	0.592	0.051	80.60%
Inception-v2	$\approx 24\text{M}$	0.638	0.612	0.050	81.51%
DenseNet-121	$\approx 8\text{M}$	0.648	0.634	0.083	82.87%
MobileNet (Baseline)	$\approx 4\text{M}$	0.645	0.636	0.080	75.20%
Portrait-NIMA (VGG-Face)	$\approx 150\text{M}$	0.778	0.785	0.045	83.41%

These statistical results confirm that integrating domain-specific embeddings (such as VGG-Face for portrait photography) substantially enhances the LCC and SRCC alignment with human perception, demonstrating the flexibility and extensibility of the

NIMA distribution-prediction engine.[27]

Works cited

1. NIMA: Neural Image Assessment - Google Research, accessed July 1, 2026, https://research.google/pubs/nima-neural-image-assessment/
2. (PDF) NIMA: Neural Image Assessment - ResearchGate, accessed July 1, 2026, https://www.researchgate.net/publication/319900846_NIMA_Neural_Image_Assessment
3. nima_enhanced_middleware_v9.4.2.py
4. Introducing NIMA: Neural Image Assessment - Google Research, accessed July 1, 2026, https://research.google/blog/introducing-nima-neural-image-assessment/
5. Image Quality Assessment Using Machine Learning - 57Blocks, accessed July 1, 2026, https://57blocks.com/blog/image-quality-assessment-using-machine-learning
6. genolve/NIMA-pytorch-aesthetic-critic: Reproduces the paper NIMA: Neural Image Assessment and adapts the model to evaluate designs generated by genetic algorithms. - GitHub, accessed July 1, 2026, https://github.com/genolve/NIMA-pytorch-aesthetic-critic
7. Using Deep Learning to automatically rank millions of hotel images | by Christopher Lennan | idealo Tech Blog | Medium, accessed July 1, 2026, https://medium.com/idealo-tech-blog/using-deep-learning-to-automatically-rank-millions-of-hotel-images-c7e2d2e5cae2
8. Quantifying Visual Image Quality: A Bayesian View - Welcome to Electrical and Computer Engineering (ECE) at the University of Waterloo, accessed July 1, 2026, https://ece.uwaterloo.ca/~z70wang/publications/ARVS_BayesianIQA.pdf
9. Neural Image Assessment (NIMA) for Image quality scores and Earth Mover's Distance | by Mehul Gupta | Data Science in Your Pocket | Medium, accessed July 1, 2026, https://medium.com/data-science-in-your-pocket/neural-image-assessment-nima-for-image-quality-scores-and-earth-movers-distance-da2cc87451c2
10. nima/NIMA Neural Image Assessment.ipynb at master · mixuala/nima - GitHub, accessed July 1, 2026, https://github.com/mixuala/nima/blob/master/NIMA%20Neural%20Image%20Ass

essment.ipynb
11. The Math Behind Word2Vec Skip-Gram Training | by Nima PourNejatian - Medium, accessed July 1, 2026, https://pournejatian.medium.com/the-math-behind-word2vec-skip-gram-implementation-ab9995730b7e
12. Quantify Image Quality Using Neural Image Assessment - MATLAB & Simulink - MathWorks, accessed July 1, 2026, https://www.mathworks.com/help/images/nima-image-quality-assessment.html
13. What is Standard Deviation? A Detailed Tutorial - Numiqo, accessed July 1, 2026, https://numiqo.com/tutorial/standard-deviation
14. Assess image quality using Siamese NIMA deep neural network in keras. - GitHub, accessed July 1, 2026, https://github.com/ryanfwy/image-quality
15. Earth mover's distance - Wikipedia, accessed July 1, 2026, https://en.wikipedia.org/wiki/Earth_mover%27s_distance
16. The Earth Mover's Distance : Formal Definition, accessed July 1, 2026, https://robotics.stanford.edu/~scohen/research/emdg/emd-defn.html
17. Computing Kantorovich-Wasserstein Distances on d-dimensional histograms using $(d+1)$-partite graphs, accessed July 1, 2026, http://papers.neurips.cc/paper/7821-computing-kantorovich-wasserstein-distances-on-d-dimensional-histograms-using-d1-partite-graphs.pdf
18. Earth Mover's Distance — dit 1.2.3 documentation, accessed July 1, 2026, https://dit.readthedocs.io/en/latest/measures/divergences/earth_movers_distance.html
19. GitHub - huangeddie/ML-Aesthetics-NIMA: A PyTorch NIMA Implementation using DenseNet. NIMA is a research endeavor that rates the aesthetic quality of images, accessed July 1, 2026, https://github.com/huangeddie/ML-Aesthetics-NIMA
20. Image Quality Prediction: Google's NIMA - Kaggle, accessed July 1, 2026, https://www.kaggle.com/code/deshwalmahesh/image-quality-prediction-google-s-nima
21. Leveraging Earth Mover's Distance Loss in XGBoost for Enhanced Predictive Performance, accessed July 1, 2026, https://medium.com/@nasdaq/leveraging-earth-movers-distance-loss-in-xgboost-for-enhanced-predictive-performance-8751e150ca3f
22. Pytorch: define custom function - python - Stack Overflow, accessed July 1, 2026, https://stackoverflow.com/questions/46509039/pytorch-define-custom-function
23. How to check relu gradient - Stack Overflow, accessed July 1, 2026, https://stackoverflow.com/questions/40623512/how-to-check-relu-gradient
24. AI回答Transformer模型中核心的MHA,FFN - Yofoo - 博客园, accessed July 1, 2026, https://www.cnblogs.com/zzz3265/p/19296485
25. Explaining Automatic Image Assessment - arXiv, accessed July 1, 2026, https://arxiv.org/html/2502.01873v1
26. Dual-Balancing for Physics-Informed Neural Networks - IJCAI, accessed July 1, 2026, https://www.ijcai.org/proceedings/2025/0797.pdf

27. Neural Aesthetic Portrait Image Assessment | CS231n - Stanford University, accessed July 1, 2026, https://cs231n.stanford.edu/2024/papers/neural-aesthetic-portrait-image-assessment.pdf
28. Flip-Robust Neural Image Assessment (FR-NIMA) for Spatially Consistent IQA - ThaiJo, accessed July 1, 2026, https://ph01.tci-thaijo.org/index.php/ecticit/article/view/264823
29. titu1994/neural-image-assessment: Implementation of NIMA - GitHub, accessed July 1, 2026, https://github.com/titu1994/neural-image-assessment